

1 Model

1.1 Model Training

The training process for the image captioning model is structured around iterative optimization of an encoder–decoder architecture, where learning progresses through multiple complete passes over the dataset, referred to as epochs. Each epoch allows the model to refine its parameters using gradient-based optimization, gradually minimizing the loss between predicted and ground-truth captions.

- **Dataset Size:** The model is trained on a total of **18,542 image–caption pairs**, where each image is associated with multiple Bengali textual descriptions. These pairs provide a rich multimodal dataset that grounds visual content in natural-language semantics.
- **Training Configuration:** Training was conducted for a maximum of **100 epochs**, with **early stopping (patience = 10)** to prevent overfitting. The training loop also applies a simple learning-rate reduction on plateau (factor = 0.5 after 3 epochs without improvement, with a floor $\text{MIN_LR} = 10^{-6}$). A batch size of $B = 16$ was used (reduced from larger sizes to avoid OOM), as implemented in the code.
- **Steps per Epoch and Total Iterations:** The number of parameter-update steps per epoch is computed in the training script using integer division: $\text{StepsPerEpoch} = \lfloor N_{\text{train}}/B \rfloor$ where N_{train} is the number of training caption samples. For $N_{\text{train}} = 18,542$ and $B = 16$ this yields

$$\text{StepsPerEpoch} = \left\lfloor \frac{18,542}{16} \right\rfloor = 1,158 \quad (1)$$

Hence the nominal total number of parameter-update operations for the configured 100 epochs is approximately:

$$\mathcal{I}_{\text{total}} = \text{StepsPerEpoch} \times \text{EPOCHS} = 1,158 \times 100 = 115,800 \quad (2)$$

Note: this matches the implementation’s use of floor division; an alternative convention is to use the ceiling of the division which would yield 1,159 steps per epoch and a slightly larger total.

- **Loss Function:** The learning objective minimizes the *sparse categorical cross-entropy* (SCCE) loss over all time steps in each generated caption. Given a target sequence of length T , the loss is defined as:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log P(y_t \mid y_{1:t-1}, I) \quad (3)$$

Here, y_t denotes the ground-truth token at step t , and $P(y_t \mid y_{1:t-1}, I)$ represents the probability assigned by the decoder given previous tokens and the encoded image I . Padding tokens are masked to ensure accurate loss computation.

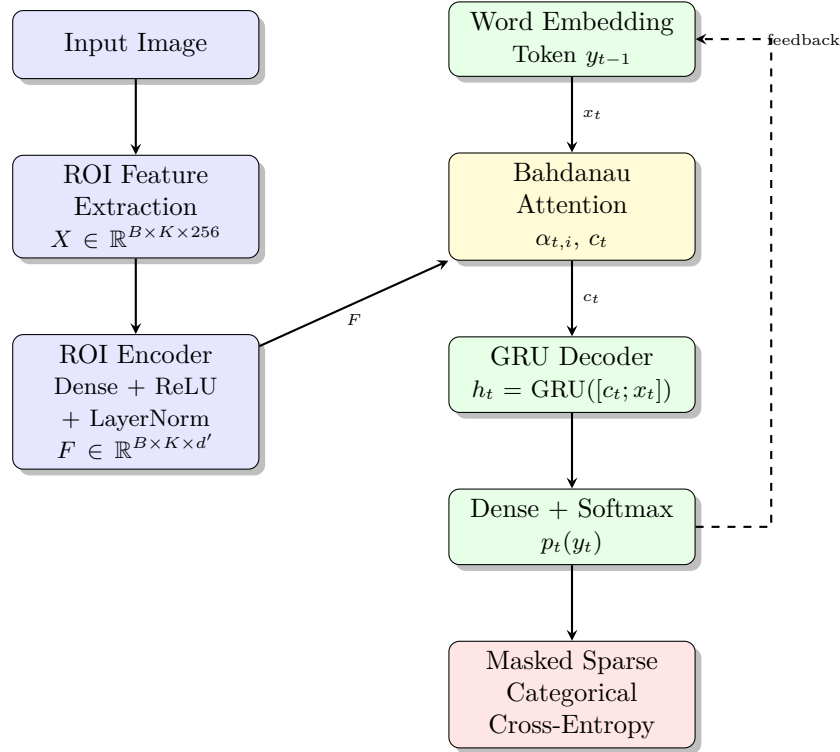
- **Optimization Strategy:** Parameter updates are performed using the **Adam optimizer**. The training procedure employs **teacher forcing**, where the decoder receives the ground-truth token at each time step, enabling faster convergence and reducing exposure bias during early training.

- **Regularization via Early Stopping:** Validation loss is monitored after every epoch. The model checkpoint corresponding to the lowest validation loss is saved automatically. Training terminates when the validation loss fails to improve for five consecutive epochs, thereby preventing overfitting and ensuring reliable generalization.
- **Training Duration:** All training was executed within the **CodeOcean environment**, with the full experiment completing in approximately **16 minutes and 24 seconds**. Model weights, tokenizer files, and experiment metadata were archived to support reproducibility and efficient re-initialization.

1.2 Model Architecture

The architecture consists of an encoder-decoder pipeline designed to translate the visual representational space of an image into a coherent Bengali textual description. The architecture has been summarized in Figure 1.

Figure 1: Encoder-Decoder Architecture with Bahdanau Attention for Bengali Image Captioning



- **Dataset:** The dataset consists of **18,542 training samples**, each containing one image accompanied by multiple Bengali descriptions.
- **Vocabulary Size:** The tokenizer yields a vocabulary of **22,276 unique words**, representing the lexical space the decoder can generate.
- **Caption Length:** The average caption length is approximately **38 tokens**, capturing detailed descriptions of image content.

1.2.1 Notation and Symbols

To improve readability we summarize the mathematical symbols used in the architecture below:

- $\mathbf{B}$: batch size (number of images per training step).
- $\mathbf{K}$: number of image regions (ROIs) per image (e.g., $K = 64$ for an 8×8 grid).
- $\mathbf{d}, \mathbf{d}'$: input ROI feature dimensionality and encoder output dimensionality respectively.
- $\mathbf{X} \in \mathbb{R}^{B \times K \times d}$: input ROI feature tensor (per-batch).
- $\mathbf{F} \in \mathbb{R}^{B \times K \times d'}$: encoder-transformed features.
- f_i : the i -th region feature vector (a length- d' vector).
- $\alpha_{t,i}$: attention weight on region i at decoding timestep t (scalar, non-negative, $\sum_i \alpha_{t,i} = 1$).
- c_t : context vector at timestep t , computed as the weighted sum $c_t = \sum_i \alpha_{t,i} f_i$.
- x_t : embedding of the previous token (input to the decoder at time t); $z_t = [c_t; x_t]$ denotes concatenation.
- h_t : decoder hidden state at timestep t ; o_t denotes the GRU output used before the final linear layer.
- p_t : probability distribution over the vocabulary at timestep t (output of softmax on logits).
- T : maximum caption length (number of timesteps considered during training/evaluation).
- m_t : binary mask (1 for real tokens, 0 for padding) used to exclude padded positions from loss/metrics.

1.2.2 Encoder: ROI Feature Encoder

The encoder maps Region-of-Interest (ROI) feature tensors into a normalized embedding space. Formally:

$$\mathbf{F} = \text{LayerNorm}(\text{ReLU}(W_{\text{enc}} \cdot \mathbf{X} + b_{\text{enc}})) \quad (4)$$

where $\mathbf{X} \in \mathbb{R}^{B \times K \times d}$ is the ROI feature tensor (with $d = 256$), and $\mathbf{F} \in \mathbb{R}^{B \times K \times d'}$ represents the transformed embeddings.

1.2.3 Attention Mechanism (Bahdanau Attention)

The decoder integrates Bahdanau attention to dynamically weight image features:

$$e_{t,i} = v^\top \tanh(W_1 f_i + W_2 h_{t-1}) \quad (5)$$

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})} \quad (6)$$

$$c_t = \sum_i \alpha_{t,i} f_i \quad (7)$$

Here, c_t denotes the context vector created by attending to salient visual regions.

1.2.4 Decoder (GRU-Based Sequence Generator)

Caption generation proceeds via a gated recurrent unit (GRU):

$$x_t = \text{Embed}(y_{t-1}), \quad z_t = [c_t; x_t] \quad (8)$$

$$h_t, o_t = \text{GRU}(z_t, h_{t-1}) \quad (9)$$

$$p_t = \text{softmax}(W_o o_t + b_o) \quad (10)$$

where p_t is the probability distribution over the next word.

1.2.5 Sparse Categorical Cross-Entropy with Masking

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T m_t \cdot \log p_t[y_t] \quad (11)$$

The mask m_t ensures that padding tokens do not contribute to the loss.

1.3 Training Algorithm

The complete training algorithm is summarized in Algorithm 1.

1.4 Evaluation Metrics

We evaluate generated captions using corpus- and image-level metrics that measure exact matches, token overlap, and n-gram/LCS-based similarity. Let the validation set contain N images. For image index i let $\mathcal{R}_i = \{r_{i,1}, \dots, r_{i,M_i}\}$ be the set of reference captions and let $\hat{y}_i$ be the single predicted caption produced by the model. Write $\text{tok}(s)$ for the token sequence of a string s , and $|\cdot|$ for the number of tokens.

Notation used in metrics

- N : number of images in the validation set.
- $\mathcal{R}_i = \{r_{i,1}, \dots, r_{i,M_i}\}$: the set of reference captions for image i .
- $\hat{y}_i$: the model's predicted caption for image i (usually the result of greedy decoding); $\text{tok}(\cdot)$ maps a string to its token sequence and $|\cdot|$ denotes token count.
- ϵ : a small positive constant added to denominators to avoid division-by-zero in metric definitions.
- C_p, C_r : multisets (counters) of tokens in the predicted caption and the chosen reference respectively; $\langle C_p \wedge C_r \rangle$ denotes the sum of per-token minima (i.e., token-wise intersection count).
- $G_n(s)$: multiset of n -grams extracted from string s ; $\text{match}_n(\hat{y}_i, r)$ counts clipped matching n -grams between prediction and reference.
- $\text{LCS}(\hat{y}_i, r)$: the length of the Longest Common Subsequence between token sequences of the prediction and reference (used in ROUGE-L computations).

Algorithm 1 Image Captioning Model Training

```
1: Initialize: encoder, decoder, optimizer (Adam), tokenizer, checkpoint managers
2: Set: best_val_loss =  $\infty$ , wait = 0, wait_lr = 0
3: Hyperparameters: EPOCHS, BATCH_SIZE, patience, LR_REDUCE_FACTOR,
   LR_REDUCE_PATIENCE, MIN_LR
4: for epoch = 1 to EPOCHS do
5:   Reset: total_train_loss = 0, total_val_loss = 0
6:   // Training Phase
7:   for each batch (img_tensor, target) in training set do
8:     hidden  $\leftarrow$  decoder.reset_state(batch_size)
9:     dec_input  $\leftarrow$   $\langle \text{start} \rangle$ ; features  $\leftarrow$  Encoder(img_tensor); loss = 0
10:    for  $t = 1$  to  $T$  do
11:      predictions, hidden, _  $\leftarrow$  Decoder(dec_input, features, hidden)
12:      loss +=  $\mathcal{L}_{\text{timestep}}(\text{target}_{:,t}, \text{predictions})$  (masked)
13:      dec_input  $\leftarrow$  target $_{:,t}$   $\triangleright$  teacher forcing
14:    end for
15:    Compute batch_total = loss/ $T$ ; apply gradients; total_train_loss += batch_total
16:  end for
17:  epoch_train_loss  $\leftarrow$  total_train_loss/num_steps
18:  // Validation Phase
19:  for each batch in validation set do
20:    Compute validation loss (no gradients); total_val_loss += batch_val_loss
21:  end for
22:  epoch_val_loss  $\leftarrow$  total_val_loss/val_steps
23:  // Metrics Computation
24:  Perform batched greedy decoding on validation set
25:  Compute metrics: accuracy, precision, recall, F1, ROUGE, token Jaccard
26:  Save metrics (CSV, plots) and generate report snippets
27:  // Checkpoint & Learning Rate Management
28:  if epoch_val_loss < best_val_loss - MIN_DELTA then
29:    best_val_loss  $\leftarrow$  epoch_val_loss; wait  $\leftarrow$  0; wait_lr  $\leftarrow$  0
30:    Save best checkpoint and model summaries
31:  else
32:    wait += 1; wait_lr += 1
33:    if wait_lr  $\geq$  LR_REDUCE_PATIENCE then
34:      lr  $\leftarrow$  max(lr  $\times$  LR_REDUCE_FACTOR, MIN_LR); wait_lr  $\leftarrow$  0
35:    end if
36:  end if
37:  Save regular checkpoint (every 5 epochs)
38:  if wait  $\geq$  patience then
39:    break  $\triangleright$  early stopping triggered
40:  end if
41: end for
42: Restore best checkpoint; save best-model metadata
```

1.4.1 Exact-match Accuracy

The exact-match accuracy counts predictions that exactly match any reference (string equality after trivial normalization):

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\hat{y}_i \in \mathcal{R}_i) \quad (12)$$

1.4.2 Per-image Token Precision and Recall and Macro Averages

For a given image i we define a per-image token precision P_i and recall R_i by comparing token counts between the predicted caption and a reference. In practice we choose the reference $r_i^* \in \mathcal{R}_i$ that maximizes an F_1 variant (best-match heuristic). Let C_p and C_r be the multisets (counters) of tokens in $\text{tok}(\hat{y}_i)$ and $\text{tok}(r_i^*)$ respectively, and write $\langle C_p \wedge C_r \rangle$ for the count of token-wise intersections. Then

$$P_i = \frac{\langle C_p \wedge C_r \rangle}{|\text{tok}(\hat{y}_i)| + \epsilon}, \quad (13)$$

$$R_i = \frac{\langle C_p \wedge C_r \rangle}{|\text{tok}(r_i^*)| + \epsilon}, \quad (14)$$

where $\epsilon > 0$ is a small constant to avoid division-by-zero. The macro-averaged precision and recall reported in the paper are the unweighted means over images:

$$\text{MacroPrecision} = \frac{1}{N} \sum_{i=1}^N P_i, \quad (15)$$

$$\text{MacroRecall} = \frac{1}{N} \sum_{i=1}^N R_i. \quad (16)$$

An image-level F_1 score can be formed as usual by $F_i = 2P_iR_i/(P_i + R_i + \epsilon)$ and macro- F_1 is the mean of F_i across images.

1.4.3 ROUGE-N

ROUGE- n compares n -gram overlap between prediction and reference. For a prediction $\hat{y}_i$ and a chosen reference r define the multiset of n -grams $G_n(\hat{y}_i)$ and $G_n(r)$. Let

$$\text{match}_n(\hat{y}_i, r) = \sum_{g \in G_n(\hat{y}_i)} \min(\text{count}_{\hat{y}_i}(g), \text{count}_r(g)), \quad (17)$$

$$P_n(\hat{y}_i, r) = \frac{\text{match}_n(\hat{y}_i, r)}{|G_n(\hat{y}_i)| + \epsilon}, \quad (18)$$

$$R_n(\hat{y}_i, r) = \frac{\text{match}_n(\hat{y}_i, r)}{|G_n(r)| + \epsilon}, \quad (19)$$

$$F_n(\hat{y}_i, r) = \frac{2P_n(\hat{y}_i, r)R_n(\hat{y}_i, r)}{P_n(\hat{y}_i, r) + R_n(\hat{y}_i, r) + \epsilon}. \quad (20)$$

When multiple references exist we adopt the common “best-reference” strategy and take the maximum F_n over $r \in \mathcal{R}_i$. The reported corpus ROUGE- n is the mean of per-image best F_n values across $i = 1 \dots N$.

1.4.4 ROUGE-L (LCS-based)

ROUGE-L uses the length of the Longest Common Subsequence (LCS) between token sequences. Define

$$\text{LCS}(\hat{y}_i, r) = \max\{\ell : \ell \text{ is length of a common subsequence between tok}(\hat{y}_i) \text{ and tok}(r)\}, \quad (21)$$

$$P_L(\hat{y}_i, r) = \frac{\text{LCS}(\hat{y}_i, r)}{|\text{tok}(\hat{y}_i)| + \epsilon}, \quad (22)$$

$$R_L(\hat{y}_i, r) = \frac{\text{LCS}(\hat{y}_i, r)}{|\text{tok}(r)| + \epsilon}, \quad (23)$$

$$F_L(\hat{y}_i, r) = \frac{2 P_L(\hat{y}_i, r) R_L(\hat{y}_i, r)}{P_L(\hat{y}_i, r) + R_L(\hat{y}_i, r) + \epsilon}. \quad (24)$$

As with ROUGE- n , we compute per-image ROUGE-L by selecting the reference $r \in \mathcal{R}_i$ that maximizes $F_L(\hat{y}_i, r)$ and report the corpus mean of these per-image F_L scores.

1.4.5 Implementation notes

All metrics above are implemented using token-level counters and dynamic-programming (for LCS) in the evaluation module. To produce stable, reproducible metric values for validation we run deterministic decoding (greedy argmax) during metric computation; sampling-based decoding can be enabled separately for diversity analysis. Per-epoch metrics are computed on the validation split and stored alongside loss histories to produce the accuracy plot and other visualization figures included in the results directory.